LAIC AI Learning Platform

Complete Specification with Frontend Workflow and Backend Logic

Ashwini Anantharaman | Life in AI Center | July 2026 | V5 Final

Introduction

This document is the complete specification for the LAIC AI Learning Platform. It covers every screen, every interaction, and every backend operation for both the instructor workflow and the student workflow. The platform allows organizations to create AI-enabled courses for any subject, whether that is neuroscience, a card game like bridge, dance, singing, sports, or any other domain. Instructors upload source material, the AI drafts a full course broken into units with all content blocks auto-generated, the instructor edits with creative control, and students learn through a structured adaptive flow with Bayesian Knowledge Tracing, six specialized AI agents, three switchable explanation modes, and a persistent AI assistant. The platform also supports community challenges where courses serve as preparation material for a larger event. Mind Bee and Brain Bee are the first two courses built on this platform, but the system is designed to be entirely generic.

Landing Page and Authentication

The landing page is the first screen any user sees. It displays the LAIC logo with an animated version of the owl mascot using a Lottie or Rive animation that plays on loop with subtle idle movements such as blinking and breathing. Below the logo are two buttons: Start Learning and Educator. Start Learning routes to the student login page. Educator routes to the instructor login page. The full UI across the platform should match the design language established in the three explanation mode frames: Interactive Mode, Summary Mode, and Narrative Mode. This means rounded corners, the indigo and purple color palette, card-based layouts, and friendly typography throughout.

Backend: Authentication

Authentication is handled by Supabase Auth. When a user clicks either button, they are routed to the appropriate login page. The login page collects email and password. On submit, the frontend calls supabase.auth.signInWithPassword. If the user does not have an account, they click Sign Up, which calls supabase.auth.signUp and creates a new row in auth.users. After successful authentication, a trigger function in Supabase automatically creates a row in the public.profiles table with the user's ID, their selected role (student or instructor), and default values for all preference fields. The role field is set based on which login page they came from. Row Level Security policies on all tables ensure that students can only read their own data and instructors can only read and write data for courses they own. The session token is stored in an HTTP-only cookie managed by the Supabase middleware in Next.js.

Instructor Workflow

Teacher Onboarding

After the instructor logs in for the first time, they are taken through a three-question onboarding flow. Question one asks what type of course they are creating. The options are Education, Card or Board Game, Dance, Singing or Music, Sports, Language, and Other with a free text field. This value is stored in the courses table as course_category and is used by every AI generation call to determine what kind of content to produce. A card game course will generate scenario-based practice questions. A dance course will generate step-by-step choreography blocks. An education course will generate traditional MCQs and textbook explanations. Question two asks who the course is for. The options are Elementary ages K through 5, Middle School ages 6 through 8, High School ages 9 through 12, College, Adult, and All Ages. This value is stored as target_age_range and controls the vocabulary level, example complexity, and tone of all AI-generated content. Question three asks the instructor's teaching style. The options are Socratic which means question-driven, Structured which means lecture-first, and Hands-on which means activity-first. This value is stored as teaching_style and changes the default block ordering and the tone of generated text. Socratic courses lead with questions before explanations. Structured courses lead with definitions. Hands-on courses lead with interactive blocks.

Backend: Onboarding

Each answer is stored by updating the instructor's row in the public.profiles table. The course_category, target_age_range, and teaching_style fields are also written to the public.courses table when the instructor creates their first course, so each course can have different settings. These three values are included in every single Claude API call as part of the structured prompt, ensuring all generated content is contextually appropriate. The onboarding flow only appears on first login. On subsequent logins, the instructor goes directly to their course homepage. The onboarding preferences can be changed later in Settings.

Upload Materials

After onboarding, the instructor is taken to the Upload Materials page. This page has a large drag-and-drop zone that accepts the following source types: PDF, DOCX, PPTX, audio files, video files, video links from YouTube or Vimeo, article URLs, website URLs, Quizlet import via URL, and Google Drive authorization to pull documents directly. The instructor can upload as many sources as they want. Each source appears as a card below the upload zone showing the filename, file type icon, upload status, and a remove button. At the bottom of the page is a Generate button that becomes active once at least one source has been uploaded.

Backend: Source Ingestion Pipeline

When a file is uploaded, it is stored in Supabase Storage under a path scoped to the course ID. A row is created in the public.source_documents table with the course_id, filename, source_type, and the raw extracted text. Text extraction depends on the source type. PDFs are parsed using the pdf-parse library. DOCX files are parsed using mammoth. PPTX files are parsed using pptx-parser. Audio and video files are sent to the OpenAI Whisper API for transcription, and the resulting transcript text is

stored. YouTube and Vimeo links are downloaded using yt-dlp, then transcribed via Whisper. Article and website URLs are scraped using a server-side fetch with HTML-to-text conversion. Quizlet URLs are parsed to extract card data as structured JSON. After text extraction, the text is chunked into segments of approximately 500 tokens with a 50 token overlap between chunks. Each chunk is then embedded using the OpenAI text-embedding-3-small model, producing a 1536-dimensional vector. The chunk text, its embedding, and metadata including the source_document_id and chunk_index are stored in the public.embeddings table, which has a pgvector column with an IVFFlat index for fast similarity search. All embeddings are tagged with the course_id so that RAG retrieval is always scoped to the current course. A Supabase RPC function called match_embeddings performs cosine similarity search filtered by course_id and returns the top K most relevant chunks for any query.

Generate Unit Outline

When the instructor clicks the Generate button on the upload page, the backend makes a Claude API call with the following context: the course_category, target_age_range, teaching_style, and a representative sample of RAG chunks from the uploaded sources, selected by querying match_embeddings with the course title as the query. The prompt instructs Claude to propose a structured unit outline with a title and one-sentence description for each unit, ordered logically with prerequisites respected. The response is parsed as JSON and each unit is written to the public.units table with the course_id, title, description, and sort_order. The instructor sees the proposed outline on screen as a list of unit cards that can be renamed, reordered via drag and drop, deleted, merged, split, or manually added to. The instructor can also click Regenerate Outline to get a new suggestion. Once the instructor is satisfied with the outline, they click Start Building to enter unit-by-unit editing.

Unit-by-Unit Block Generation

The instructor is taken to Unit 1. The AI auto-generates all content blocks for this unit. The default generated blocks in order are: a hook text block containing a curiosity question or surprising fact to open the unit, a main content text block containing the core explanation adapted to the teaching style and explanation mode, an animation or simulation description block containing a visual explainer if applicable to the subject, a video clip block if a relevant video was uploaded which is auto-cropped to the relevant timestamp section, a flashcards block containing key terms and definitions from this unit, a quiz block containing four to six questions whose type depends on the course category, and a reflection prompt block containing one metacognition question. Each block is displayed as a card in a vertical list. Each card shows the block type label with a colored tag indicating its step category such as Discover, Build Intuition, Study, Practice, or Reflect. Each card also shows an AI Generated badge, the preview of the generated content, an edit button that opens an inline editor, a delete button, a regenerate button that asks the AI to create a new version, and a drag handle for reordering. At the top of the unit editing page, there are three checkboxes for explanation modes: Real World, Conversational, and Textbook. The instructor checks which modes they want the AI to generate for the main content text block. If all three are checked, the AI generates three separate versions of the main content, each stored as a separate block with the explanation_mode field set accordingly. At the top of the page there is also a plus button that lets the instructor manually add a new block of any type

including text, flashcards, MCQ, scenario, open response, video embed, article embed, animation, audio record prompt, video record prompt, or reflection. On the right side of the page is a live preview panel that shows a miniature version of what the student will see for this unit, updating in real time as the instructor edits. There is also an AI chat at the bottom of the editing page. The instructor can type instructions like make the hook question more engaging or add a flashcard for the term synaptic vesicle, and the AI edits the relevant block. An icon next to the chat input shows which block the AI is currently focused on. When the instructor is done editing Unit 1, they click the Next arrow to move to Unit 2. The AI generates all blocks for Unit 2 using the same process. This repeats for every unit in the course.

Backend: Block Generation

For each unit, the backend makes multiple Claude API calls, one per block type. Each call includes the course_category, target_age_range, teaching_style, unit_title, unit_number and total_units for sequencing context, the explanation_mode if applicable, and the top five to eight RAG chunks retrieved by querying match_embeddings with the unit title as the search query. The Claude response for each block type is structured as JSON. For text blocks, the response contains a content string. For flashcards, the response contains an array of term and definition pairs. For MCQ blocks, the response contains the question text, an array of options, the correct index, and an array of four hints at progressive levels. For animation blocks, the response contains a natural language description of the visual and optionally a GSAP configuration object. For reflection blocks, the response contains a prompt string. Each generated block is written to the public.blocks table with the unit_id, block_type, explanation_mode, step category, content stored as JSONB, sort_order, is_interactive flag, ai_generated set to true, and instructor_approved set to false. When the instructor edits a block, the content JSONB is updated and instructor_approved is set to true. When the instructor clicks regenerate, a new Claude API call is made for that specific block and the content is replaced.

Explanation Modes

The platform supports three explanation modes for content text blocks. Real World mode uses everyday examples and analogies to explain concepts. The icon is a globe emoji. Conversational mode uses informal, friendly language as if a friend is explaining it. The icon is a lightbulb emoji. Textbook mode uses formal, precise academic language. The icon is a book emoji. During course creation, the instructor checks which modes to enable per unit. The AI generates a separate version of the main content text for each enabled mode. Each version is stored as a separate block in the database with the explanation_mode field set. On the student side, the student's preferred mode from onboarding is their default. Three icons appear in the top right corner of the content area. The active mode icon is filled. Available but inactive mode icons are outlined. Modes the instructor did not implement are grayed out. The student can tap any available icon to switch, and the content block swaps to that mode's version. Every mode switch is logged to the public.mode_switches table with the student_id, block_id, from_mode, and to_mode. This data feeds the teacher dashboard to show which modes students are switching away from, indicating which explanations are not working.

Final Preview and Publishing

After the instructor has reviewed all units, they see a full course preview page. This page shows all units in order with all blocks within each unit rendered as the student would see them. The explanation mode switcher is functional in the preview so the instructor can check all three versions. The page also shows the total block count and estimated completion time. The instructor can click any unit to go back and edit it, reorder units, or add and remove units. When satisfied, the instructor clicks Publish. The backend updates the course status in the public.courses table from draft to published. Students can now enroll. After publishing, the instructor can edit any unit or block at any time. Changes take effect immediately without needing to unpublish. The backend simply updates the relevant rows in the blocks table.

Instructor Sidebar Navigation

Lessons

The Lessons button on the instructor sidebar takes the instructor to the unit editing view described above. This is where all course content is created and managed. The instructor can navigate between units using next and previous arrows or by clicking a unit in the sidebar list.

Instructor Dashboard

The Dashboard button takes the instructor to a page showing detailed analytics for their course. At the top is an alert banner showing any students who have not logged in for more than five days. Below that are three summary cards: active students this week, average mastery percentage, and number of new misconceptions detected. Below the cards is a student table sorted by who is furthest behind, showing the student name, their weakest unit, their overall mastery percentage, when they were last active, and an active or inactive badge. Below the table is the misconception log, showing the most common misconceptions ranked by frequency. Each misconception shows the misconception text, which unit it came from, how many students have it, and a Fix button that routes directly to the editor for the relevant block. At the bottom of the page, the AI generates structural suggestions based on aggregate learner data. Examples include suggesting that a unit be made optional if skip data shows no downstream impact, or recommending that the instructor add a Socratic dialogue block to address a specific misconception. Each suggestion has an Accept button that auto-applies the change and a Dismiss button.

Backend: Dashboard Data

The dashboard data is served by a single API route at GET /api/dashboard/[courseId]. This route queries multiple tables and returns aggregated data. Active students are counted by querying quiz_responses and reflections for entries within the last seven days grouped by student_id. Average mastery is calculated by averaging the p_known field from learner_mastery for all students enrolled in the course. Misconceptions are queried from the public.misconceptions table filtered by course_id, grouped by misconception_text, and ordered by frequency descending. The student table is built by joining profiles, learner_mastery, and quiz_responses to compute per-student mastery and last active timestamp, then sorting by mastery ascending so the most struggling students appear first. AI suggestions are generated by a Planner Agent call that receives the aggregate mastery data, misconception patterns, and mode switch data, and returns structured suggestions as JSON.

Challenge Details

The Challenge Details button leads to a page with three tabs. The Students Registered tab shows a list of all enrolled students with their name, email, enrollment date, and current mastery. The instructor can manually add a student by email. There is also a Generate Link button that creates a unique classroom join link and a Generate Code button that creates a short alphanumeric classroom code. Both are stored in the public.courses table. The Info tab shows the challenge name, description, challenge date with a countdown timer, and any other relevant fields that the instructor can edit. The Add Teachers tab

shows a teacher join link that other instructors can use to join the course as co-instructors under the same organization. When a teacher joins via this link, their profile is added to a public.course_instructors junction table with the course_id and their user_id.

Settings

The Settings button has two tabs. The Organization Details tab shows the instructor's login email, display name, organization name, and the ability to change their password via supabase.auth.updateUser. The AI Prompts tab shows all current system prompts used by the AI agents, organized by grade level. Each prompt is displayed in an editable text area. When the instructor saves a modified prompt, it is stored in a public.custom_prompts table with the course_id, agent_name, grade_level, and the custom prompt text. On all future AI generation calls for that course, the backend checks for custom prompts before using the defaults. This allows instructors to fine-tune how the AI teaches their specific course without needing to understand the underlying architecture.

Student Workflow

Student Login and Onboarding

When a student clicks Start Learning on the landing page, they are taken to the student login page. After logging in or signing up, first-time students are taken through an onboarding flow. The onboarding asks five questions. Question one asks the student's age or grade level. Question two asks about prior knowledge in the subject. Question three asks their learning goal, with options such as challenge preparation, general curiosity, school enrichment, or exam prep. Question four asks their preferred explanation mode, presenting the three options Real World, Conversational, and Textbook with icons and descriptions. The student selects one as their default. Question five asks their preferred session length, with options like 10 minutes, 20 minutes, 30 minutes, or open-ended. All answers are stored in the public.profiles table. The preferred explanation mode is stored as preferred_explanation_mode and becomes the default content view for every unit. The session length is stored as session_length_minutes and controls how many blocks the Planner Agent includes in a single session recommendation.

Backend: Student Onboarding

After onboarding completes, the backend also initializes the student's learner model. For every unit in every course the student enrolls in, a row is created in public.learner_mastery with the student_id, unit_id, mastery_state set to not_seen, p_known set to 0.0, attempts set to 0, and correct set to 0. This ensures the Planner Agent and the mastery dashboard have data to work with from the very first session.

Courses Homepage

After onboarding, the student is taken to their courses homepage. This page shows cards for each course the student is enrolled in, with the course title, a progress bar showing overall mastery, and a last studied timestamp. There is a plus button that opens a modal with two options: paste a join link or enter a classroom code. When the student submits either, the backend verifies the link or code against the public.courses table. If valid, a row is created in public.enrollments with the student_id and course_id, and the learner model is initialized for all units in that course. The student then sees the course appear on their homepage and can click it to enter the student interface.

Student Sidebar Navigation

The student interface has a sidebar with five buttons: Lessons, Student Dashboard, Challenge Details, Tools, and Settings.

Lessons: The Core Learning Experience

The Lessons button takes the student to the full lesson view. The left sidebar shows all units with mastery dots next to each, colored green for mastered, yellow for in progress, and gray for not yet seen. The student clicks a unit to view it. Each unit is a single scrollable page with all blocks that the instructor generated displayed in order. This is the key difference from StudyFetch, which separates

quizzes, flashcards, chat, and video into different tabs. On this platform, everything is integrated into one continuous flow per unit: the hook text, the animation or simulation, the study content, the flashcards, the practice questions, and the reflection prompt all appear on the same page in sequence.

At the top right of the content area are three explanation mode icons: globe for Real World, lightbulb for Conversational, and book for Textbook. The student's default mode from onboarding is active. The student can tap any available icon to switch. If the instructor did not implement a mode for this unit, that icon is grayed out. A separate icon, the interactive icon, appears if an animation or simulation block exists for this unit. If no interactive block exists, this icon is also grayed out. When the student switches modes, the main content text block swaps to the new mode's version. All other blocks, including flashcards, quizzes, animations, and reflections, remain the same regardless of mode.

The three modes render the same content differently. In Summary Mode, the content is presented in a newspaper-style layout with columns, a headline-style unit title, and structured paragraphs. In Narrative Mode, the content uses the same newspaper-style layout but the text is written in a storytelling tone as if narrating the concept through a real scenario. In Interactive Mode, the text content is presented as a text messaging conversation between the AI tutor and the student. The messages appear in chat bubbles as if the tutor is explaining the concept in a back-and-forth dialogue. All non-text blocks such as flashcards, animations, videos, and quizzes remain in their standard format regardless of which mode is selected.

Practice blocks are embedded inline in the lesson flow. Multiple choice questions show four options. When the student selects an answer, the response is sent to the Quiz Coach Agent. If incorrect, the progressive hint system activates. There are four hint levels: level one is a nudge such as think about what you learned about X, level two is a concept hint such as this relates to the concept of Y, level three is a directional hint such as the answer involves Z because, and level four is a full explanation without giving the direct answer. Each hint is revealed one at a time on button click. The number of hints used is recorded. Flashcard blocks display inline with flip functionality. The student can mark each card as known or needs review. At the end of the unit, a reflection prompt block appears with an open text area. The student must submit a reflection before moving to the next unit. The student clicks the Next arrow to proceed to the next unit.

Backend: Lesson Rendering and AI Agents

When a student opens a unit, the frontend fetches all blocks for that unit from the public.blocks table, filtered by unit_id and ordered by sort_order. Blocks are filtered by the student's current explanation_mode. If the student switches modes, the frontend re-fetches blocks with the new mode filter and logs the switch to public.mode_switches. The AI Assistant sidebar is rendered as a persistent panel on the right side of every lesson page. The student can type a question at any time. The question is sent to the Student Assistant Agent at POST /api/agents/assistant along with the current unit_id, block_id, and explanation_mode as context. The agent retrieves relevant RAG chunks scoped to the course, and responds with a streamed answer. The agent's system prompt includes a guardrail instruction: never give direct quiz answers, redirect to hints instead. The agent can also surface past reflections by querying the public.reflections table for the student and referencing relevant entries in its

response.

When a student submits a quiz response, the answer is sent to the Quiz Coach Agent at POST /api/agents/quiz. The agent evaluates whether the answer is correct, identifies any misconceptions in the student's reasoning, and returns structured JSON with the evaluation result, feedback text, and any detected misconception. The backend then performs three database operations. First, it inserts a row into public.quiz_responses with the student_id, block_id, the response, whether it was correct, hints used, and time spent. Second, it calls the BKT update function. The updateBKT function takes the current p_known value from learner_mastery and the correct boolean, applies the Bayesian update formula using the four BKT parameters (pInit 0.1, pTransit 0.2, pSlip 0.1, pGuess 0.25), and returns the new p_known value. The mastery_state field is then updated based on the new p_known: below 0.1 is not_seen, 0.1 to 0.3 is exposed, 0.3 to 0.6 is practiced, 0.6 to 0.85 is understood, and above 0.85 is mastered. Third, if a misconception was detected, a row is inserted or updated in public.misconceptions with the student_id, unit_id, and misconception_text. When a student submits a reflection, the response is sent to the Reflection Agent at POST /api/agents/reflect. The agent assesses the depth of the reflection on a scale of 0 to 1. If the depth is below 0.3, the agent generates a follow-up probe question. The reflection text is embedded using the OpenAI embeddings API and stored in public.reflections with the embedding vector for future retrieval by the assistant agent.

AI Assistant During Video

When a unit contains a video block, the video player renders with a transcript sidebar on the right. The transcript is displayed as clickable lines with timestamps. The current playback position is highlighted in the transcript. The video progress bar shows small markers at timestamps where AI-generated pause points occur. At each pause point, the video pauses automatically and a reflection banner appears asking a comprehension question. The student answers before the video resumes. Below the transcript is an AI chat input. The student can ask questions about the video content at any point. The Student Assistant Agent receives the question along with the video's transcript as additional RAG context. When the agent references a specific part of the video, it includes a clickable timestamp link in its response. Clicking the link seeks the video to that timestamp. The transcript for each video is stored in the source_documents table and is chunked and embedded into the course's RAG namespace, so the agent can retrieve specific passages from the video when answering questions.

Student Dashboard

The Student Dashboard tab takes the student back to their course page with a detailed progress view. This page shows a mastery bar for each unit using the five-state color coding: gray for not seen, light blue for exposed, yellow for practiced, orange for understood, and green for mastered. Below the unit list is a summary showing total units completed, total quizzes taken, overall mastery percentage, and a list of flagged misconceptions with links back to the relevant units. The Planner Agent generates a personalized next unit recommendation displayed as a card at the top of the page, explaining why this unit was chosen based on the student's current mastery state and prerequisite dependencies.

Backend: Student Dashboard

The student dashboard data is served by GET /api/mastery/[studentId]. This route queries learner_mastery for all units in the student's enrolled courses, computes aggregate statistics, and returns them as JSON. The Planner Agent call at POST /api/agents/planner receives the full learner model (all mastery states for all units) and the prerequisite graph (the prerequisites array on each unit), and returns the recommended next unit ID with a one-sentence explanation.

Challenge Details

The Challenge Details tab shows a countdown timer to the challenge date, the student's current course mastery percentage, and a button labeled Practice Material. Clicking Practice Material routes to the Tools tab. When the student completes all units and achieves mastery on all required units, a certificate is generated. The certificate is a PDF created on the backend using a template with the student's name, the course title, the completion date, and the organization name. It is stored in Supabase Storage and displayed as a downloadable link on this page.

Tools

The Tools button opens a popup with five options: View Quizzes, View Tests, View Flashcards, Audio, and Notes. View Quizzes shows all quiz blocks compiled across all units. Quizzes from units the student has not yet reached are locked. Each quiz shows the unit it belongs to, the number of questions, and whether it has been completed. There is a Generate More button on the bottom right that triggers the Quiz Coach Agent to create additional practice questions for the student's weakest units based on their mastery data. View Tests works the same way but with longer assessments combining questions across multiple units. View Flashcards shows all flashcard blocks compiled across all units, with locked cards for unreached units. The Generate More button creates additional flashcards focused on terms the student has missed in quizzes. The Audio tab shows video and audio content organized by unit, each with a transcript on the right, AI pause point markers on the timeline, and an AI chat on the bottom right. The Notes tab shows AI-generated summary notes per unit with a chatbot on the right that answers questions about the notes. All practice in quizzes, tests, and flashcards follows Bayesian Knowledge Tracing. Every response updates the BKT model. Questions the student gets wrong are recycled at increasing intervals using a spaced repetition schedule driven by the p_known value.

Settings

The student Settings tab has accessibility options including font size adjustment, high contrast mode for color blind users, deaf and hard of hearing settings that ensure all audio content has visible transcripts and captions, reduced motion settings that disable animations, and screen reader compatibility settings. These preferences are stored in the public.profiles table and applied via CSS classes and conditional rendering on the frontend.

Agentic Architecture

The platform uses six specialized AI agents, each implemented as a TypeScript module in the lib/agents directory. Each agent is an async function that receives a shared AgentContext object and returns a structured response. The AgentContext includes the student_id, course_id, unit_id, current block, explanation mode, mastery state, misconceptions array, recent reflections, session history, and the student's profile including age range, learning goal, preferred mode, and session length. All agents use the Claude API with structured system prompts. The Orchestrator Agent routes every incoming user interaction to the correct agent by classifying intent: is the user asking for help, answering a quiz, reflecting, or browsing. The Concept Tutor Agent delivers Socratic teaching by retrieving RAG chunks for the current unit and generating a dialogue that adapts to the selected explanation mode. It never gives definitions first and always asks a question before explaining. The Quiz Coach Agent generates questions dynamically based on the unit content and the student's mastery state, evaluates responses, updates BKT, and logs misconceptions. The Reflection Agent generates metacognition prompts and assesses the depth of student reflections, probing further if the response is shallow. The Planner Agent reads the full learner model and sequences units via the prerequisite graph, generating a personalized next-session recommendation. It also generates structural suggestions for the teacher dashboard based on aggregate learner patterns. The Student Assistant Agent powers the persistent sidebar, providing context-aware on-demand help. Its system prompt explicitly instructs it to never give direct quiz answers and to redirect to hints instead.

Adaptive Engine and Bayesian Knowledge Tracing

The adaptive engine tracks genuine mastery per unit per student using Bayesian Knowledge Tracing. Each unit for each student has a p_known probability value that starts at 0 and updates after every quiz response. The BKT model uses four parameters: pInit at 0.1 representing the probability of initial mastery, pTransit at 0.2 representing the probability of learning on each attempt, pSlip at 0.1 representing the probability of getting an answer wrong despite mastery, and pGuess at 0.25 representing the probability of guessing correctly without mastery. After each response, the posterior probability of knowing is calculated using Bayes theorem, and then the transition probability is applied. The resulting p_known value maps to one of five mastery states: not_seen below 0.1, exposed from 0.1 to 0.3, practiced from 0.3 to 0.6, understood from 0.6 to 0.85, and mastered above 0.85. If a student is struggling with a unit, meaning their p_known drops or stays below 0.3 after multiple attempts, the Planner Agent detects this and adapts the session. It may reinforce the missed content by generating additional practice in different formats such as flashcards, animations, or scenario questions, or it may suggest the student revisit a prerequisite unit. The adaptive engine also tracks explanation mode switch patterns. If a student switches away from a particular mode on a specific unit, this is logged and aggregated on the teacher dashboard, signaling to the instructor that the explanation in that mode may not be effective.

Content Generation by Course Category

The course category selected during instructor onboarding changes the type of content the AI generates, not the underlying system. For education courses, the main content is textbook-style explanations with examples, practice consists of MCQs and open response questions, and animations show concept diagrams and process flows. For card and board game courses, the main content is rules explanations with hand or board examples, practice consists of scenario-based questions such as you have this hand what do you bid, and animations show game state visualizations. For dance courses, the main content is step-by-step movement descriptions with counts, practice consists of watch and identify questions and video record prompts, and animations show formation diagrams and timing visualizations. For singing and music courses, the main content is technique explanations with reference audio, practice consists of audio record prompts and pitch or rhythm identification, and animations show waveform visualizations and musical notation. For sports courses, the main content is strategy and technique breakdowns, practice consists of scenario-based decision making, and animations show play diagrams and position visualizations. The block types audio_record and video_record allow students to submit recordings that the AI evaluates, enabling practice for performance-based courses without requiring specialized engines. These specialized engines such as game engines, body trackers, or pitch detectors can be integrated later as embeddable blocks via iframe using a future game_embed block type.

Animation and Simulation Pipeline

The platform supports two types of visual content. Explain mode produces pre-rendered animations using the GSAP and SVG pipeline. The instructor types a concept description in the animation block editor. The backend sends this description along with RAG chunks for the unit to the Claude API with a two-stage prompt. Stage one asks Claude to create a structured scene plan describing what visual elements appear, in what order, what labels are shown, and what the narrative arc is. Stage two asks Claude to convert the scene plan into a GSAP animation configuration as JSON, specifying SVG elements, their initial states, and the GSAP timeline commands. The frontend renders this configuration as a live SVG animation in the browser. No video files are generated. Explore mode produces interactive simulations using GSAP, D3.js, or Three.js. The instructor selects a primitive from the library such as a slider, particle system, graph plotter, 3D viewer, or state diagram, and sets its parameters through a form. The primitive renders in the browser with full interactivity. Students can drag sliders, change values, and explore. Both animation types are stored as JSONB in the blocks table under the content field, with the block_type set to animation or interactive_simulation and the is_interactive flag set accordingly.

Course Self-Improvement Loop

The platform creates a closed feedback loop where courses improve over time based on real learner data. After a course is published and students begin learning, the teacher dashboard surfaces patterns: which units have low mastery, which misconceptions are most common, which explanation modes students switch away from, where students drop off or stall, and which quiz questions are too easy or too hard. The instructor can click directly from the dashboard into the editor to fix any issue. The AI also

generates proactive suggestions. If aggregate data shows that students who skip a particular unit perform equally well on downstream quizzes, the AI suggests making that unit optional. If a significant number of students switch from the textbook mode to the real world mode on a specific unit, the AI suggests improving the textbook explanation or deprioritizing it. New source material uploaded after initial publication is processed through the same ingestion pipeline and enriches the course's RAG namespace. This means the AI assistant and tutor agents get smarter over time as more context becomes available. The course does not need to be rebuilt to incorporate new sources. The instructor simply uploads additional material and the RAG is updated automatically.

Technical Stack

The frontend is built with Next.js 14 using the App Router, styled with Tailwind CSS and shadcn/ui components. Authentication and the database are handled by Supabase, which provides PostgreSQL with the pgvector extension for embedding storage, Supabase Auth for email and OAuth authentication, Supabase Storage for file uploads, and Row Level Security for data isolation. The LLM layer uses the Claude API with the claude-sonnet-4-6 model for all content generation, agent responses, and evaluation. Embeddings are generated using the OpenAI text-embedding-3-small model. Video and audio transcription uses the OpenAI Whisper API. The RAG framework uses LlamaIndex for TypeScript for document loading, chunking, and retrieval orchestration. The six AI agents are implemented as plain TypeScript functions with typed inputs and outputs, orchestrated by a simple state machine without requiring a framework like LangGraph. Animations use GSAP for 2D browser animations, D3.js for data-driven visualizations, and Three.js for 3D viewers. The application is deployed on Vercel for the frontend with Supabase handling all backend infrastructure. The Lottie or Rive library is used for the mascot character animations throughout the UI.